From Prompts to Trees: Effective LLM-Guided Tree Generation for Few-Shot Tabular Classification

Yue Qiu[1*], **Zekang Du**[1*], **Yiqun Diao**[2], **Bingsheng He**[2], **Qinbin Li**[1†]
[1]School of Computer Science and Technology, Huazhong University of Science and Technology
[2]School of Computing, National University of Singapore
{yueqiu, zekangdu, qinbin}@hust.edu.cn,
yiqun@comp.nus.edu.sg, dcsheb@nus.edu.sg

Abstract

While Large Language Models (LLMs) possess rich world knowledge and impressive generalization capabilities, their direct application to tabular data classification is hindered by high inference costs and limited interpretability. In contrast, decision trees are fast and transparent but often underperform in low-data regimes. In this work, we propose a novel framework that bridges these paradigms by distilling LLM knowledge into interpretable decision trees under a few-shot learning setting. Instead of directly prompting the LLM to generate full trees, which is often unstable and inefficient, we develop a three-stage paradigm that prompts the LLM to generate rules and organize the rules into a tree. Experiments on multiple real-world tabular datasets demonstrate that our method achieves superior accuracy and interpretability with significantly lower prompting overhead compared to existing baselines.

1 Introduction

Large Language Models (LLMs) (OpenAI, 2023; Touvron et al., 2023; DeepSeek-AI et al., 2024; Naveed et al., 2025; Bai et al., 2023) have demonstrated remarkable capabilities in various domains, including code generation, planning, and complex problem-solving tasks. Despite these advancements, their potential remains significantly underexplored in the context of traditional tabular data classification (Shwartz-Ziv and Armon, 2022; Borisov et al., 2024). Tabular data is still the widely used format of data in reality, and is ubiquitous in critical domains such as healthcare diagnostics, financial risk assessment, and customer behavior prediction.

Although LLMs inherently possess extensive knowledge, their direct application to tabular data classification faces two prominent challenges: (1) limited interpretability, making it difficult to provide intuitive justifications for classification decisions, and (2) inefficiency in inference, which restricts their applicability in resource-constrained and real-time scenarios. Decision trees (Loh, 2011; Chen and Guestrin, 2016; Ke et al., 2017; Rokach and Maimon, 2014; Quinlan, 1986, 1993), on the other hand, are widely adopted for tabular data classification due to their inherent interpretability, high computational efficiency, and ability to capture non-linear relationships between features. It has been a dominant model for tabular data classification for a long period. However, traditional tree training requires substantial labeled data to capture non-linear feature interactions. While LLMs can extract prior semantic knowledge from textified features to compensate for this data scarcity, combining them as a knowledge source and an efficient executor remains a challenge in practical few-shot settings.

This raises an important question: *can we present the knowledge of LLMs by decision trees for tabular data classification in few-shot settings?* If successful, such a tree representation would simultaneously inherit the LLM's rich prior knowledge while retaining the interpretability and fast inference of decision trees, thereby addressing the key limitations of directly applying LLMs to tabular classification. A straightforward approach is to prompt the LLM once to directly generate a tree to represent its knowledge. However, this yields unreliable results due to the inherent complexity and instability of long-context reasoning. Another method is to generate the reasoning process by prompting multiple times. For example, Tree-of-Thought (Yao et al., 2023) approaches involve recursively prompting LLMs to generate the reasoning paths in a tree structure. Nevertheless, such methods suffer from substantial prompting overhead when applied to tabular data classification, rendering them impractical for wide adoption.

∗ Equal contribution.
† Corresponding author.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 13451–13481
October 24-29, 2026 ©2026 Association for Computational Linguistics

In this work, we propose **LLMT**, a modular and efficient framework that distills LLM knowledge into decision trees through a three-stage pipeline: (1) *Rule Generation*, where the LLM produces structured, machine-parseable classification rules; (2) *Tree Assembly*, where these rules are organized into a tree based on Gini impurity computed from a small labeled dataset; and (3) *Leaf Refinement*, where the LLM corrects noisy leaf decisions. Unlike prompting-based methods such as Chain-of-Thought (CoT) and Tree-of-Thought (ToT), which construct a reasoning chain or tree in a single pass or through recursive expansion, LLMT breaks the task into distinct stages. Our experiments show that this modular design improves both stability and efficiency.

Our contributions are as follows:

- We introduce a **novel three-stage distillation paradigm** that bridges the gap between expensive, opaque LLM reasoning and efficient, interpretable tree models.
- We provide a **formal generalization analysis** of LLMT under explicit structural assumptions, clarifying its statistical properties.
- We conduct **extensive experiments** on 11 datasets with 17 baselines, reporting accuracy and efficiency across different settings.
- We show that LLMT achieves consistent and significant accuracy improvements, large token-cost savings, and interpretable outputs in the few-shot setting.

2 Background and Related Work

2.1 Decision Trees

Decision trees (Loh, 2011; Rokach and Maimon, 2014; Quinlan, 1986, 1993) are among the most fundamental models in machine learning, widely used in tasks requiring interpretable decision-making. A decision tree forms a hierarchy of if–then rules: internal nodes test features, branches denote test outcomes, and leaves assign class labels. Trees are learned by recursively selecting splits that reduce impurity, such as Gini impurity (Loh, 2011).

2.2 LLMs for Tabular Data

Tabular data is one of the most prevalent formats in real-world applications and can be naturally converted into text by enumerating feature-value pairs. Recent work has explored LLMs for tabular prediction via fine-tuning, instruction tuning, synthetic data generation, or direct prompting (Wang et al., 2024; Herzig et al., 2020; Zhang et al., 2023; Hegselmann et al., 2023; Jaitly et al., 2023; Fang et al., 2024; Dinh et al., 2022; Wen et al., 2024). TabLLM (Hegselmann et al., 2023) fine-tunes T0 (Sanh et al., 2022) with a few tabular examples, while InsightTab (Yuan et al., 2025) and SumBoost (Manikandan et al., 2023) use LLMs directly as tabular classifiers with improved prediction pipelines. Although effective, these methods rely on LLMs during inference, leading to limited interpretability and high prediction cost.

Several recent works further investigate whether LLMs can generate rule-based or tree-like structures for tabular data (Knauer et al., 2025; Carrasco et al., 2025; Xiong et al., 2024). Direct-ZSDT (Knauer et al., 2025) and Step-ZSDT (Carrasco et al., 2025) build decision trees zero-shot but cannot use labeled examples to guide construction. GPTree (Xiong et al., 2024) uses LLMs to pre-generate rule candidates from raw training samples, making it more suitable for data-rich settings but less robust in few-shot regimes where limited or noisy examples may bias rule generation.

Other related methods combine LLMs with downstream estimators. FeatLLM (Han et al., 2024) uses LLMs for feature engineering before training a linear model, while DeLTa (Ye et al., 2025) uses LLM-refined rules to guide residual error correction in tree ensembles. In contrast, our method aims to generate a standalone decision tree: it first extracts structured rules from the LLM's prior knowledge and then organizes them statistically using few-shot examples, achieving interpretability, low inference cost, and data-aware tree construction.

2.3 Prompting with Structured Reasoning

Few-shot prompting is a widely adopted strategy to elicit reasoning capabilities from LLMs using only a handful of labeled examples. Prior work (Wei et al., 2022; Yao et al., 2023; Ning et al., 2024; Besta et al., 2024) has shown that making the intermediate reasoning process explicit in the prompt significantly enhances LLM performance. Chain-of-Thought (CoT) prompting (Wei et al., 2022) introduces the idea of decomposing complex tasks into a sequence of intermediate reasoning steps, leading to better accuracy.

Code is available at `github.com/yueqiu0/LLMTree`.

Features	LLMT	Direct-ZSDT	Step-ZSDT	CoT-Tree	ToT-Tree	IO-Tree	TabLLM	InsightTab	SumBoost	FeatLLM	DeLTa	GPTree
Few-shot ready (no finetune)	✓	✗	✗	✓	✓	✓	✗	✓	✓	✓	✓	✓
Token-efficient build	✓	✓	✗	✓	✗	✓	✗	✗	✗	✗	✗	✗
Tree generation	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✓
No LLM at inference	✓	✓	✓	✓	✓	✓	✗	✗	✗	✓	✓	✓
Rule-first (vs path-first)	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✓
Sample-level privacy preservation	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗
Theoretical generalization bound	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

Table 1: Comparison of our LLM-guided tree method (**LLMT**) with other related studies

Building on this, Tree-of-Thoughts (ToT) (Yao et al., 2023) generalizes CoT by considering multiple branching reasoning paths, enabling more robust and explorative problem-solving. Skeleton-of-Thought (SoT) (Ning et al., 2024) further refines this approach by prompting the LLM to first generate a high-level reasoning structure and then elaborate on each component in parallel.

Positioning. Existing approaches fall into three categories: (i) using LLMs directly as in-context tabular classifiers (Hegselmann et al., 2023; Yuan et al., 2025; Manikandan et al., 2023); (ii) using LLMs to generate features, rules, or residual corrections for downstream models (Han et al., 2024; Ye et al., 2025); and (iii) prompting LLMs to generate reasoning paths or tree-like structures (Wei et al., 2022; Yao et al., 2023; Xiong et al., 2024). LLMT differs from these by separating knowledge extraction from tree construction: it extracts structured rules from feature semantics, assembles them statistically with few-shot labeled examples, and then performs leaf refinement. Unlike data-driven rule-generation methods such as GPTree (Xiong et al., 2024), LLMT does not rely on many raw training samples to induce rules, making it better suited to few-shot settings. Moreover, LLMT preserves sample-level privacy by deriving rules solely from feature semantics without transmitting raw data values to the LLM. Our design avoids using the LLM as the final classifier and yields an explicit decision tree with interpretability and low inference cost. Table 1 summarizes the comparison.

3 Motivation

We start by conducting experiments to gain insights into the performance and capabilities of LLMs in the context of tabular data classification. Our observations highlight both the strengths and limitations of LLMs, motivating our proposed approach.

Observation 1: LLMs have rich background knowledge for tabular data classification.

We assess the predictive performance of four LLMs on public tabular datasets under two scenarios: 1) *original*, where we convert the tabular data into text and explicitly provide feature names and task descriptions in the query prompts; 2) *masked*, where feature names and task descriptions are omitted (see Appendix F for details.) The experimental results on two public tabular datasets Diabetes (Smith et al., 1988) and Spambase (Hopkins et al., 1999), summarized in Figure 1 under zero-shot prompting conditions, clearly show that the prediction accuracy of LLMs substantially deteriorates when metadata is removed. To minimize the potential influence of pretraining memorization, we extend this comparison to six newer datasets, with complete results reported in Table 7 of Appendix F.1.

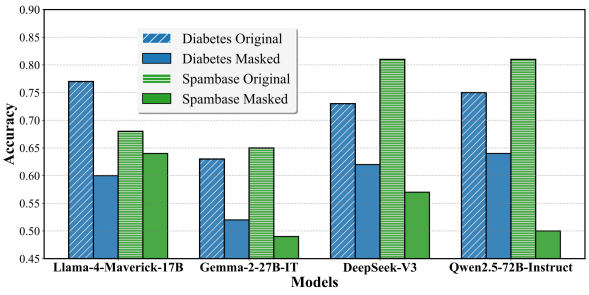
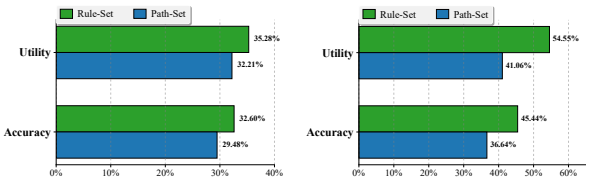


Figure 1: The test accuracy of LLMs with or without the metadata about the dataset.



(a) Diabetes (b) Spambase

Figure 2: Comparison of Utility and Accuracy between generated rule sets and path sets

Observation 2: Compared to generating long reasoning paths, LLMs are better at producing discrete rules for tabular data classification.

Recent studies (Han et al., 2024) suggest that LLMs excel at feature engineering. To further investigate this capability, we conduct a comparative analysis of LLM performance in generating long

reasoning paths versus discrete rules for tabular data classification. Reasoning paths can be seen as a series of rule-checks that traverse from the root to a leaf node in the decision tree. For a controlled comparison, we prompt LLMs to generate reasoning-path sets and discrete-rule sets containing the same total number of atomic feature conditions. We then evaluate the generated sets using Set Utility and Set Accuracy across eight datasets (see Appendix F.2 for detailed settings and complete results). The representative results, shown in Figure 2, reveal that rule generation achieves higher Set Utility and Set Accuracy than path generation. These findings indicate that while LLMs struggle to produce coherent and complete hierarchical reasoning paths in few-shot settings, they are particularly effective at generating accurate and interpretable individual classification rules.

These findings motivate our design choice: rather than asking the LLM to produce full reasoning paths, which are often unstable, lengthy, and error-prone, we instead focus on *discrete rule generation and organization*.

4 Methodology

4.1 Problem Statement

We consider the problem of distilling the knowledge embedded in an LLM into an interpretable and efficient decision tree for tabular data classification. Formally, let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ denote a tabular dataset with n samples, where each input $x_i \in \mathbb{R}^m$ is a feature vector of m attributes and $y_i \in \{1, 2, \ldots, K\}$ is a categorical label over K classes. The goal is to learn a decision tree T such that $T(x)$ approximates the classification capability of the LLM while maintaining interpretability and low computational overhead.

Crucially, we assume access to only a small number of labeled samples (few-shot setting) and the ability to query an LLM via prompting. This constraint reflects practical scenarios where labeled tabular data is scarce but the model's background knowledge is rich.

4.2 Framework Overview

To tackle the above challenge, we propose a novel three-stage distillation framework named **LLMT** that efficiently extracts and structures the LLM's latent decision-making knowledge into a decision tree. Instead of prompting the LLM to generate the reasoning paths as a tree, we ask it to generate discrete rules and then organize them as a tree in a statistical way. As illustrated in Figure 3, our framework comprises the following components: (1) **Rule Generation**: We prompt the LLM with carefully designed instructions to generate discrete classification rules, along with a self-assessed confidence score for each rule. (2) **Tree Assembly**: We systematically organize the induced rules into a tree structure by selecting high-quality rules in a layer-wise manner using Gini-based splitting criteria on the small labeled dataset. (3) **Leaf Refinement**: We prompt the LLM to analyze each rule path, evaluate whether rules are satisfied, and assign confidence scores to possible labels, creating a probability distribution that quantifies prediction certainty. Please refer to Appendix D for an example of the execution process.

Implementation Details. We adopt a level-wise *heap indexing* scheme: the root is indexed as 1, its left and right children as 2 and 3, and so on in breadth-first order. At each node, we consider rules not yet used along the current path. Candidate rules are grouped by their confidence scores into multiple bins, processed in descending order. Within each group, ties are broken by highest Gini gain and lowest index to ensure determinism. The process terminates when either the maximum tree depth is reached or no remaining rule yields a positive gain.

4.3 Rule Generation

In the first stage, we prompt the LLM to generate a structured set of candidate rules. Each rule follows the grammar:

$$f_j \ \mathsf{op} \ \theta \ ,$$

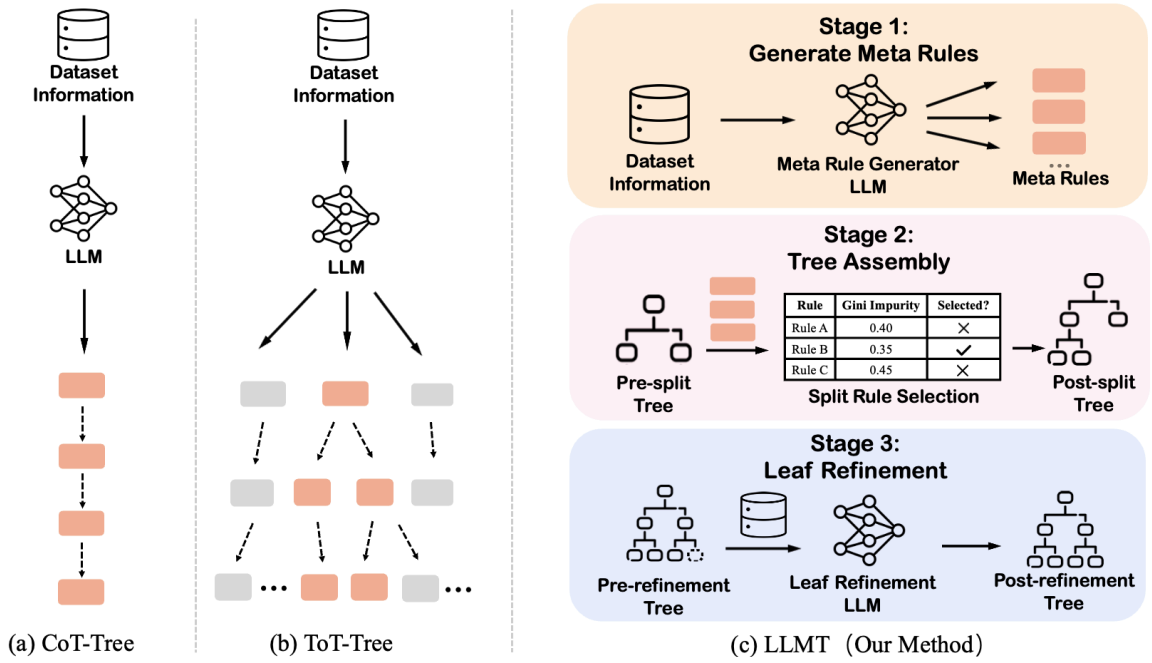
where $\mathsf{op} \in \{<, \geq, =, \neq\}$, θ is a numerical threshold or categorical value, and f_j represents the j-th feature in the dataset (e.g., age ≥ 20). Categorical features are encoded using one-vs-rest splits.

The LLM receives (i) a *system prompt* detailing task constraints and rules, and (ii) a *dataset prompt* containing feature names, types, and dataset-agnostic rule set examples for formatting.

The model outputs rules with self-assessed confidence scores in $[1, 10]$.

4.4 Tree Assembly

The objective of this stage is to construct a decision tree T using the rule set $\mathcal{R}$ and few-shot examples $\mathcal{D}_{\text{train}}$. Our idea is to put the important rules on the top of the tree so that they can be shared in more reasoning paths, while using the knowledge in few-shot examples to adjust the node positions.



(a) CoT-Tree (b) ToT-Tree (c) LLMT (Our Method)

Figure 3: Comparison between our method and prompting-based methods for generating reasoning trees. Instead of (1) asking the LLM to directly generate long reasoning paths (CoT-Tree), or (2) asking the LLM to generate the next thought based on a long reasoning path, our method prompts the LLM to generate discrete rules, organizes them into a tree using statistical methods, and then uses the LLM for final refinement.

Rule Grouping and Selection. The process proceeds in a level-wise fashion through the tree. For each node at depth level l, we exclude rules already used on the path R_{path} to that node. We then group the remaining candidates by their confidence scores (rules with the same confidence value form one group). Starting with the highest-confidence group, we select the optimal rule R_k by minimizing the total Gini impurity of the resulting splits. Specifically, suppose the current tree node corresponds to a subset $\mathcal{D}_p \subseteq \mathcal{D}_{\text{train}}$ of samples. Then, we aim to find a rule R_k as the node $T[i]$ such that

$$\arg\min_{R_k} \frac{|\mathcal{D}_l^k|}{|\mathcal{D}_p|} I_G(\mathcal{D}_l^k) + \frac{|\mathcal{D}_r^k|}{|\mathcal{D}_p|} I_G(\mathcal{D}_r^k), \quad (1)$$

where $\mathcal{D}_l^k$ is the left child set split by R_k and $\mathcal{D}_r^k$ is the right child set split by R_k. Intuitively, the following proposition holds:

Proposition 4.1 (Expected Impurity Reduction)**.** *Let $r^* \in \mathcal{R}$ be the selected rule at node t, splitting dataset $\mathcal{D}_p \subseteq \mathcal{D}_{train}$ into $\mathcal{D}_l, \mathcal{D}_r$. Then, the expected impurity after splitting satisfies:*

$$\mathbb{E}[\text{Gini}(\mathcal{D}_l, \mathcal{D}_r)] \leq \text{Gini}(\mathcal{D}_p),$$

with equality if and only if the split yields no information gain.

4.5 Leaf Refinement

While the internal structure of the tree is built from data-driven rule selection, the final classification decisions at leaf nodes may still be noisy or misaligned with LLM's original knowledge. To address this, we introduce a refinement step that evaluates and corrects the leaf nodes if necessary.

For every path p from the root to a leaf in T, we construct a natural language query describing the rule chain and prompt the LLM to evaluate its logical soundness. The LLM returns confidence scores $\mathbf{C_p}$ (a probability distribution over all possible labels). We identify the label y^* with the highest confidence score c^*. If $c^* > \tau$ (a predefined threshold) and y^* differs from the current leaf label, we adjust the classification at that leaf node to y^* to improve consistency and generalization. As we show in Section 5.5, our leaf refinement process usually improves the accuracy of the tree. Using the PAC theory (Blumer et al., 1989), we have the following theorem for the learned tree:

Theorem 4.2 (Generalization Error Bound)**.** *Consider a decision tree T constructed using LLMT of maximum depth d from a finite rule set $\mathcal{R}$ with size K. Given n training samples, for any $\delta > 0$, with probability at least $1 - \delta$, the generalization error*

$R(T)$ *satisfies:*

$$R(T) \leq \hat{R}(T) + \sqrt{\frac{2\left(\log P(K, 2^d - 1) + \log \frac{2}{\delta}\right)}{n}} \quad (2)$$

$$\leq \hat{R}(T) + \sqrt{\frac{2\left((2^d - 1)\log K + \log \frac{2}{\delta}\right)}{n}}, \quad (3)$$

where $P(K, 2^d - 1) = K!/(K - (2^d - 1))!$.

The proof is available at Appendix A. While this PAC-style bound can be numerically loose in the extreme few-shot regime (n is small), it provides crucial structural guidance. The bound explicitly shows that tree depth d is the dominant complexity driver (via the 2^d dependence), justifying our shallow tree design. Moreover, by distilling the LLM into a finite rule set K, LLMT induces a finite, statistically analyzable hypothesis space, distinguishing it from mathematically intractable black-box prompting pipelines. As shown in our experiments (Appendix G.3), LLMT consistently outperforms baseline methods across varying tree depths.

4.6 Ensemble Extension

For high-dimensional datasets, we optionally extend LLMT into an ensemble termed LLMT Forest, since ensembles can improve model capacity while reducing the overfitting risk of a single deep tree (Breiman, 2001; Chen and Guestrin, 2016). To construct LLMT Forest, we randomly permute and partition the features into disjoint, approximately equal-sized subsets, train one LLMT tree on each subset using the same few-shot samples, and aggregate their predictions by majority vote.

5 Evaluation

We report representative results in the main paper. In the Appendix, we also include results on more baselines, more datasets and different tree depths.

5.1 Experimental Setup

Datasets We evaluate our method on 11 widely used public tabular datasets including **Nursery** (Rajkovic, 1989), **Diabetes** (Smith et al., 1988), **Spambase** (Hopkins et al., 1999), **Abalone** (Nash et al., 1994), **Blood** (Yeh, 2008), **Iris** (Fisher, 1936), **Breast** (Patrício et al., 2018), **Glioma** (Tasci et al., 2022), **Communities** (Redmond, 2002), **Ecom** (Rubachev et al., 2025), and **Myocardial** (Golovenkin et al., 2020). Among them, we select Diabetes, Spambase, and Nursery as the major datasets for evaluation across various settings. These datasets span diverse domains and vary widely in sample size and feature dimensionality, providing a comprehensive testbed.

Baselines We compare LLMT against the following baselines, organized by methodological category: (1) *zero-shot decision-tree methods*: **DirectZSDT** (Knauer et al., 2025), **StepZSDT** (Carrasco et al., 2025); (2) *conventional machine-learning methods*: **LogReg** (logistic regression; Cox, 1958), **SVM** (support vector machine; Cortes and Vapnik, 1995), and **CART** (Loh, 2011); (3) *tree-ensemble methods*: **Random Forest** (Breiman, 2001), **XGBoost** (Chen and Guestrin, 2016), **CatBoost** (Prokhorenkova et al., 2018), and **LightGBM** (Ke et al., 2017); (4) *LLM prompting methods*: **IO-Tree**, **CoT-Tree** (Wei et al., 2022), and **ToT-Tree** (Yao et al., 2023); (5) *pretrained black-box predictors*: **TabPFN** (Hollmann et al., 2025) and **TabLLM** (Hegselmann et al., 2023); and (6) *recent LLM-assisted tabular learning methods*: **FeatLLM** (Han et al., 2024), **DeLTa** (Ye et al., 2025), and **GPTree** (Xiong et al., 2024). Due to space constraints, additional comparisons involving Random Forest, CatBoost, TabPFN, and TabLLM are reported in Appendix G.

Settings Experiments are run on a Linux server with 4× Intel Xeon Gold 5117 CPUs and 4× Nvidia Tesla V100 GPUs. We use Qwen2.5-72B-Instruct via TogetherAI (Together AI, 2024) by default, and provide all prompts in the Appendix. We set the maximum tree depth to 3 or 4 depending on the dataset, and set the number of rules as $K = \max(10, 2^d - 1)$, where d is the maximum depth. The leaf-refinement threshold is $\tau = 0.7$ for all datasets. We repeat each experiment for 10 trials with randomly sampled training sets and a fixed test set of 100 samples, and report mean ± standard deviation. Additional details are in Appendix F.

5.2 Effectiveness

We evaluate LLMT against all baselines in terms of classification accuracy. Table 2 reports the results with #shots=2 per class, while Table 3 reports results under varying numbers of training examples on three representative datasets.

LLMT consistently outperforms almost all baselines, with more than 10% improvement over the strongest baseline in many settings. It also achieves robust gains over recent LLM-assisted methods such as FeatLLM, DeLTa, and GPTree: FeatLLM

Dataset	DirectZSDT	StepZSDT	LogReg	SVM	CART	XGBoost	LightGBM	IO-Tree	CoT-Tree	ToT-Tree	FeatLLM	DeLTa	GPTree	LLMT
Nursery	0.485	0.450	0.558	0.564	0.602	0.434	0.578	0.439	0.461	0.353	0.571	0.448	0.479	**0.762**
Diabetes	0.729	0.641	0.618	0.629	0.546	0.690	0.543	0.539	0.701	0.561	0.700	0.541	0.616	**0.764**
Spambase	0.645	0.732	0.790	0.783	0.723	0.600	0.733	0.723	0.739	0.727	0.802	0.747	0.661	**0.811**
Abalone	0.550	0.526	0.684	0.672	0.689	0.670	0.658	0.647	0.694	0.570	**0.721**	0.656	0.667	0.694
Blood	0.597	0.371	0.604	0.635	0.565	0.605	0.508	0.585	0.500	0.551	0.642	0.550	0.572	**0.675**
Iris	**0.946**	0.547	0.793	0.756	0.882	0.739	0.685	0.870	0.903	0.867	0.903	0.823	0.814	0.940
Breast	0.554	0.500	0.597	0.607	0.500	0.400	0.513	0.543	0.473	0.593	0.544	0.529	0.523	**0.617**
Glioma	0.504	0.743	**0.760**	0.755	0.592	0.560	0.680	0.637	0.598	0.652	0.724	0.643	0.678	**0.760**
Average	0.626	0.564	0.676	0.675	0.637	0.587	0.612	0.623	0.634	0.609	0.701	0.617	0.626	**0.753**

Table 2: Accuracy comparison of different methods when the number of training examples per class is two. Best performances are bolded, and our method's performances, when second-best, are underlined.

Datasets	**Nursery**					**Diabetes**					**Spambase**				
#Shots	3	6	12	24	48	2	4	8	16	32	2	4	8	16	32
DirectZSDT	0.485	0.485	0.485	0.485	0.485	**0.729**	0.729	0.729	0.729	0.729	0.645	0.645	0.645	0.645	0.645
StepZSDT	0.450	0.450	0.450	0.450	0.450	0.641	0.641	0.641	0.641	0.641	0.732	0.732	0.732	0.732	0.732
LogReg	0.505	0.558	0.670	0.778	**0.855**	0.542	0.618	0.674	0.730	0.748	0.685	0.790	0.785	0.782	0.835
SVM	0.500	0.564	0.661	0.781	0.833	0.542	0.629	0.660	0.671	0.710	0.685	0.783	0.754	0.771	0.807
CART	0.351	0.602	**0.786**	0.803	0.801	0.519	0.546	0.624	0.663	0.686	0.639	0.723	0.743	0.705	0.757
XGBoost	0.460	0.434	0.744	0.784	0.808	0.690	0.690	0.626	0.708	0.708	0.600	0.600	0.706	0.706	0.762
LightGBM	0.378	0.578	0.780	0.792	0.803	0.600	0.543	0.631	0.676	0.680	0.652	0.733	0.759	0.717	0.769
IO-Tree	0.378	0.439	0.519	0.511	0.516	0.525	0.539	0.605	0.649	0.573	0.661	0.723	0.671	0.743	0.711
CoT-Tree	0.407	0.461	0.617	0.559	0.591	0.665	0.701	0.700	0.696	0.733	0.699	0.739	0.705	0.740	0.731
ToT-Tree	0.379	0.353	0.357	0.370	0.411	0.645	0.561	0.583	0.563	0.551	0.673	0.727	0.722	0.695	0.720
FeatLLM	0.539	0.571	0.717	0.761	0.781	0.658	0.700	0.717	0.725	0.730	0.769	0.802	0.814	0.813	0.837
DeLTa	0.357	0.448	0.577	0.673	0.729	0.310	0.541	0.624	0.637	0.630	0.394	0.747	0.747	0.811	0.823
GPTree	0.326	0.479	0.780	0.741	0.720	0.515	0.616	0.646	0.643	0.594	0.641	0.661	0.677	0.746	0.644
LLMT (Ours)	**0.762**	**0.762**	0.784	**0.818**	0.840	0.662	**0.764**	**0.770**	**0.770**	**0.770**	**0.782**	**0.811**	**0.836**	**0.839**	**0.843**

Table 3: Mean accuracy over ten runs across varying #shots (standard deviation is available at Appendix G). DirectZSDT and StepZSDT are constant across #shots since they cannot utilize any training examples.

and DeLTa tend to overfit in extreme few-shot regimes, while GPTree can be affected by data-induced bias due to its reliance on large-scale training data. These results show the benefit of leveraging LLM knowledge in a structured, data-aware manner. In addition, ToT-Tree often performs worse than CoT-Tree, suggesting that recursive node-level prompting introduces noise and inconsistency for structured tabular data. By contrast, LLMT uses LLM-generated rules as decision units, leading to more effective tree construction.

We further assess high-dimensional performance on three datasets with over 100 attributes. These results show that LLMT maintains its advantage in high-dimensional feature spaces and further support the effectiveness of its ensemble extension. Please refer to Appendix G.3 for detailed results.

5.3 Efficiency

We evaluate computational efficiency by measuring training time and token usage per prediction on three tabular datasets, as summarized in Table 4. We omit CART, XGBoost, and IO-Tree because they are already highly efficient but have relatively poor classification performance. DirectZSDT incurs high latency due to its two-stage dialogue, where the LLM first generates a tree and then converts it into executable Python code. StepZSDT is even more expensive, as it prompts the LLM to generate feature-specific rules for each node.

Compared with CoT-Tree, LLMT achieves comparable training time and token usage. Compared with ToT-Tree, LLMT avoids repeated planning and voting, reducing token usage by 10–22$\times$ and construction time by 1.6–3.6$\times$. LLMT also has comparable computational cost to DeLTa, while being up to 10$\times$ faster than FeatLLM and more token-efficient than GPTree, which rely on repeated rule-parsing queries and sequential batch-wise insight generation, respectively.

5.4 Interpretability

Decision trees are intrinsically interpretable because each prediction follows an explicit root-to-leaf path, making the decision process directly inspectable. Compared with coefficient-based, hy-

	Time (s)			Token (#Input token, #Output token, #Total)		
	Diabetes	Nursery	Spambase	Diabetes	Nursery	Spambase
DirectZSDT	43.64	31.75	55.33	(2.9k, 1.1k, 4k)	(3.8k, 1.3k, 4.1k)	(3.4k, 1.4k, 4.8k)
StepZSDT	1006.67	1191.40	4036.18	(85.6k, 22.1k, 107.7k)	(128.3k, 27.2k, 155.5k)	(358.9k, 95.4k, 454.3k)
CoT-Tree	17.00	9.58	12.05	(2.7k, 1.3k, 4k)	(2.7k, 1.0k, 3.7k)	(5.6k, 0.9k, 6.5k)
ToT-Tree	33.39	33.22	64.56	(32.9k, 0.5k, 33.4k)	(35.4k, 0.6k, 36k)	(140k, 1.3k, 141.3k)
FeatLLM	259.07	416.16	324.06	(22.2k, 10.3k, 32.5k)	(24.4k, 16.3k, 40.7k)	(48.5k, 12.3k, 60.8k)
DeLTa	30.20	26.90	28.84	(3.1k, 3.3k, 6.4k)	(3.1k, 2.8k, 5.9k)	(5.2k, 3.1k, 8.3k)
GPTree	140.75	103.47	125.56	(12.5k, 2.3k, 14.8k)	(9.5k, 3.2k, 12.7k)	(27.6k, 2.5k, 30.0k)
LLMT	20.48	16.33	18.18	(3.1k, 0.2k, 3.3k)	(3.4k, 0.2k, 3.6k)	(6.0k, 0.4k, 6.4k)
Savings	**1.63x**	**2.03x**	**3.56x**	**10.1x**	**10x**	**22.1x**

Table 4: The training time (s) and #token in the #shots=32 setting. The savings of LLMT are computed against ToT-Tree in terms of #total tokens.

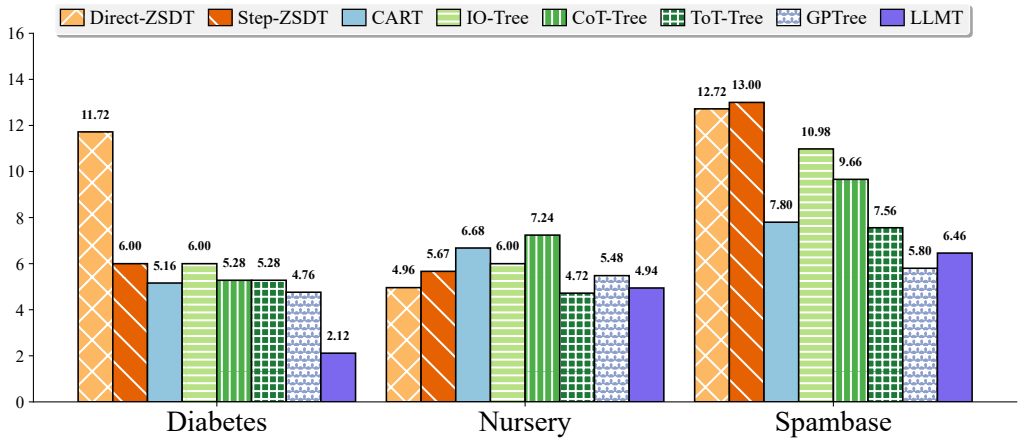


Figure 4: Average number of nodes in trees generated by each method. Fewer nodes indicate a more concise tree.

Dataset	**Direct-ZSDT**	**Step-ZSDT**	**CART**	**IO-Tree**	**CoT-Tree**	**ToT-Tree**	**GPTree**	**LLMT**
Diabetes	0.422	0.420	0.414	0.397	0.397	0.403	0.425	**0.396**
Nursery	0.665	0.634	0.516	0.582	0.534	0.629	0.552	**0.456**
Spambase	0.378	0.478	0.360	**0.341**	0.361	0.351	0.396	0.352
Average	0.488	0.511	0.430	0.440	0.431	0.461	0.458	**0.401**

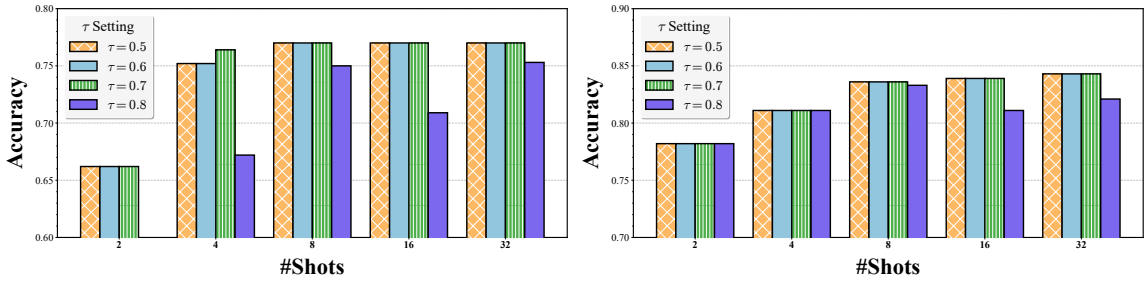
Table 5: Average Gini impurity of rules across different methods. Lower values indicate better rule quality.

brid, or black-box models, standalone trees generally provide more intuitive explanations for individual predictions. We therefore compare methods that output standalone trees in terms of structural conciseness and split purity, measured by average node count and average Gini impurity, as reported in Figure 4 and Table 5, respectively.

LLMT produces the most concise trees on Diabetes and remains among the most compact methods on Nursery and Spambase. It also achieves the lowest average Gini impurity, performing best on Diabetes and Nursery and close to the best method on Spambase. Overall, LLMT retains the intrinsic readability of a standalone decision tree while generating compact structures with high-quality rules.

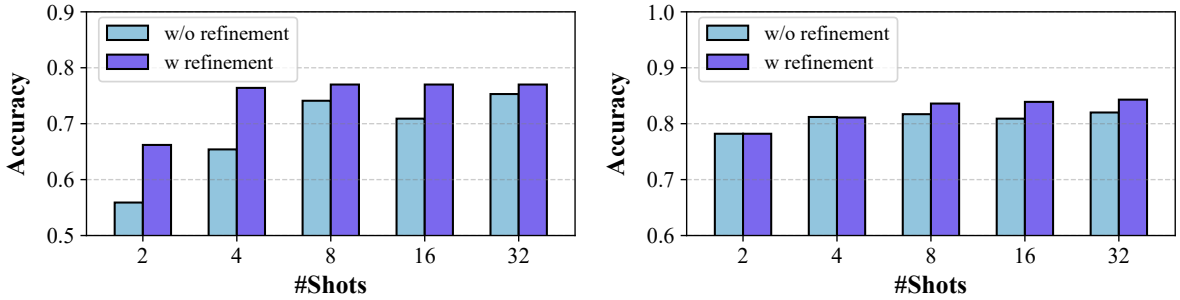
5.5 Ablation and Sensitivity Study

We further analyze the effect of the *Leaf Refinement* stage and its confidence threshold τ. The threshold τ controls whether the LLM accepts a leaf node's decision based on its confidence in the corresponding reasoning path. As shown in Fig-



(a) Diabetes (b) Spambase

Figure 5: The accuracy of LLMT with different τ



(a) Diabetes (b) Spambase

Figure 6: The accuracy of LLMT with/without leaf refinement

ure 5, LLMT is generally stable across different τ values, and $\tau = 0.7$ usually achieves the best performance, suggesting a good balance between preserving the initial tree structure and applying LLM-based corrections. It indicates that LLMT does not require extensive hyperparameter tuning.

We then remove *Leaf Refinement* to evaluate its contribution on Diabetes and Spambase. Figure 6 shows consistent accuracy gains of 8% and 2%, respectively. Beyond accuracy, LLM verification may identify same-label sibling leaves for merging, simplifying the tree and mitigating spurious splits caused by limited few-shot data.

6 Conclusion

In this paper, we present a novel framework for distilling the knowledge of LLMs into interpretable and efficient decision trees for tabular data classification. Unlike prior approaches that prompt LLMs to directly generate full decision structures which often leads to instability and high inference costs, our method utilizes the ability of LLMs to generate discrete rules and organizes them as a tree statistically. By bridging the strengths of LLMs and symbolic models, our method offers a promising direction for building trustworthy AI systems that can reason over structured data.

Limitations

Despite its strengths, our method assumes that the LLM possesses sufficient background knowledge relevant to the target dataset. If the dataset lies outside the knowledge scope of the LLM, then the generated rules—and consequently the constructed tree—may be inaccurate or ineffective. Moreover, our method inherits any biases in the underlying LLM, which may influence the rules extracted and affect downstream decisions. Caution must be exercised when deploying the distilled trees in high-stakes applications, and future work should explore fairness-aware prompting and post-hoc auditing.

Acknowledgements

This research is supported in part by National Natural Science Foundation of China (Grant No. 62502174). This research is also supported in part by the National Research Foundation, Singapore and Infocomm Media Development Authority under its Trust Tech Funding Initiative. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Infocomm Media Development Authority.

References

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, and 29 others. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609*.

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690.

Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth. 1989. Learnability and the Vapnik-Chervonenkis dimension. *Journal of the ACM*, 36(4):929–965.

Vadim Borisov, Tobias Leemann, Kathrin Seßler, Johannes Haug, Martin Pawelczyk, and Gjergji Kasneci. 2024. Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(6):7499–7519.

Leo Breiman. 2001. Random forests. *Machine Learning*, 45(1):5–32.

Lucas Carrasco, Felipe Urrutia, and Andrés Abeliuk. 2025. Zero-shot decision tree construction via large language models. *arXiv preprint arXiv:2501.16247*.

Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794. ACM.

Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Machine Learning*, 20(3):273–297.

D. R. Cox. 1958. The regression analysis of binary sequences. *Journal of the Royal Statistical Society: Series B (Methodological)*, 20(2):215–232.

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, and 181 others. 2024. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*.

Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Michael Gira, Shashank Rajput, Jy-yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. 2022. LIFT: Language-interfaced fine-tuning for non-language machine learning tasks. In *Advances in Neural Information Processing Systems*, volume 35, pages 11763–11784. Curran Associates, Inc.

Xi Fang, Weijie Xu, Fiona Anting Tan, Jiani Zhang, Ziqing Hu, Yanjun Qi, Scott Nickleach, Diego Socolinsky, Srinivasan Sengamedu, and Christos Faloutsos. 2024. Large language models (LLMs) on tabular data: Prediction, generation, and understanding—a survey. *Transactions on Machine Learning Research*.

R. A. Fisher. 1936. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188.

S. E. Golovenkin, V. A. Shulman, D. A. Rossiev, P. A. Shesternya, S. Yu. Nikulina, Yu. V. Orlova, and V. F. Voino-Yasenetsky. 2020. Myocardial infarction complications. UCI Machine Learning Repository. Dataset.

Sungwon Han, Jinsung Yoon, Sercan O. Arik, and Tomas Pfister. 2024. Large language models can automatically engineer features for few-shot tabular learning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 17454–17479. PMLR.

Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. 2023. TabLLM: Few-shot classification of tabular data with large language models. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 5549–5581. PMLR.

Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. 2020. TaPas: Weakly supervised table parsing via pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4320–4333. Association for Computational Linguistics.

Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. 2025. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326.

Mark Hopkins, Erik Reeber, George Forman, and Jaap Suermondt. 1999. Spambase. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C53G6X.

Muhammad Kamal Hossen and Mohammad Shorif Uddin. 2025. Students suspicious behaviors detection dataset for AI-powered online exam proctoring. Mendeley Data.

Sukriti Jaitly, Tanay Shah, Ashish Shugani, and Razik Singh Grewal. 2023. Towards better serialization of tabular data for few-shot classification with large language models. *Preprint*, arXiv:2312.12464.

Kaggle. 2025a. Predict the introverts from the extroverts. Playground Series, Season 5, Episode 7.

Kaggle. 2025b. Predicting loan payback. Playground Series, Season 5, Episode 11.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pages 3146–3154. Curran Associates, Inc.

Ricardo Knauer, Mario Koddenbrock, Raphael Wallsberger, Nicholas M. Brisson, Georg N. Duda, Deborah Falla, David W. Evans, and Erik Rodner. 2025. "Oh LLM, I'm asking thee, please give me a decision tree": Zero-Shot decision tree induction and embedding with large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1196–1206. ACM.

Wei-Yin Loh. 2011. Classification and regression trees. *WIREs Data Mining and Knowledge Discovery*, 1(1):14–23.

Hariharan Manikandan, Yiding Jiang, and J. Zico Kolter. 2023. Language models are weak learners. In *Advances in Neural Information Processing Systems*, volume 36, pages 50907–50931.

Warwick Nash, Tracy Sellers, Simon Talbot, Andrew Cawthorn, and Wes Ford. 1994. Abalone. UCI Machine Learning Repository. Dataset.

Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. 2025. A comprehensive overview of large language models. *ACM Transactions on Intelligent Systems and Technology*, 16(5):1–72. Article 106.

Xuefei Ning, Zinan Lin, Zixuan Zhou, Zifu Wang, Huazhong Yang, and Yu Wang. 2024. Skeleton-of-thought: Prompting LLMs for efficient parallel generation. In *The Twelfth International Conference on Learning Representations*.

OpenAI. 2023. GPT-4 technical report. *Preprint*, arXiv:2303.08774.

Miguel Patrício, José Pereira, Joana Crisóstomo, Paulo Matafome, Manuel Gomes, Raquel Seiça, and Francisco Caramelo. 2018. Using resistin, glucose, age and BMI to predict the presence of breast cancer. *BMC Cancer*, 18(1):29.

Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. CatBoost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31, pages 6638–6648. Curran Associates, Inc.

J. Ross Quinlan. 1986. Induction of decision trees. *Machine Learning*, 1(1):81–106.

J. Ross Quinlan. 1993. *C4.5: Programs for Machine Learning*, first edition. Morgan Kaufmann.

Vladislav Rajkovic. 1989. Nursery. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5P88W.

Michael Redmond. 2002. Communities and crime. UCI Machine Learning Repository. Dataset.

Lior Rokach and Oded Maimon. 2014. *Data Mining with Decision Trees: Theory and Applications*, second edition, volume 81. World Scientific.

Ivan Rubachev, Nikolay Kartashev, Yury Gorishniy, and Artem Babenko. 2025. TabReD: Analyzing pitfalls and filling the gaps in tabular deep learning benchmarks. In *International Conference on Learning Representations*, volume 2025, pages 35166–35202.

Victor Sanh, Albert Webson, Colin Raffel, Stephen H. Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, Manan Dey, M. Saiful Bari, Canwen Xu, Urmish Thakker, Shanya Sharma, Eliza Szczechla, Taewoon Kim, Gunjan Chhablani, Nihal V. Nayak, and 23 others. 2022. Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*.

Ravid Shwartz-Ziv and Amitai Armon. 2022. Tabular data: Deep learning is not all you need. *Information Fusion*, 81:84–90.

Jack W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes. 1988. Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. In *Proceedings of the Annual Symposium on Computer Application in Medical Care*, pages 261–265. PMCID: PMC2245318.

Muthukumaran Subramaniyan. 2023. Paddy dataset. UCI Machine Learning Repository. Dataset.

Erdal Tasci, Ying Zhuge, Harpreet Kaur, Kevin Camphausen, and Andra Valentina Krauze. 2022. Hierarchical voting-based feature selection and ensemble learning model scheme for glioma grading with clinical and molecular characteristics. *International Journal of Molecular Sciences*, 23(22):14155.

Together AI. 2024. Together.ai. `https://www.together.ai/`.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, and 49 others. 2023. Llama 2: Open foundation and fine-tuned chat models. *Preprint*, arXiv:2307.09288.

Zifeng Wang, Chufan Gao, Cao Xiao, and Jimeng Sun. 2024. MediTab: Scaling medical tabular data predictors via data consolidation, enrichment, and refinement. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pages 6062–6070. International Joint Conferences on Artificial Intelligence Organization.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc.

Xumeng Wen, Han Zhang, Shun Zheng, Wei Xu, and Jiang Bian. 2024. From supervised to generative: A novel paradigm for tabular deep learning with large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 3323–3333. ACM.

Sichao Xiong, Yigit Ihlamur, Fuat Alican, and Aaron Ontoyin Yin. 2024. GPTree: Towards explainable decision-making via LLM-powered decision trees. *arXiv preprint arXiv:2411.08257*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822.

Hangting Ye, Jinmeng Li, He Zhao, Dandan Guo, and Yi Chang. 2025. LLM meeting decision trees on tabular data. In *Advances in Neural Information Processing Systems*, volume 38, pages 130884–130920.

I-Cheng Yeh. 2008. Blood transfusion service center. UCI Machine Learning Repository. Dataset.

Yifei Yuan, Jiatong Li, Weijia Zhang, Mohammad Aliannejadi, Evangelos Kanoulas, and Renjun Hu. 2025. Summarize-exemplify-reflect: Data-driven insight distillation empowers LLMs for few-shot tabular classification. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 12324–12348.

Tianping Zhang, Shaowen Wang, Shuicheng Yan, Li Jian, and Qian Liu. 2023. Generative table pre-training empowers models for tabular prediction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14836–14854. Association for Computational Linguistics.

A Proof

Proposition 4.1 (Expected Impurity Reduction)**.** *Let $r^* \in \mathcal{R}$ be the selected rule at node t, splitting dataset D into D_l, D_r. Then, the expected impurity after splitting satisfies:*

$$\mathbb{E}[\mathrm{Gini}(D_l, D_r)] \leq \mathrm{Gini}(D),$$

with equality if and only if the split yields no information gain.

Proof. Define the Gini impurity for dataset D with C classes as:

$$\mathrm{Gini}(D) = 1 - \sum_{i=1}^{C} p_i^2, \quad p_i = \frac{|D^{(i)}|}{|D|}.$$

The selected rule r^* minimizes:

$$\mathbb{E}[\mathrm{Gini}(D_l, D_r)] = \frac{|D_l|}{|D|}\mathrm{Gini}(D_l) + \frac{|D_r|}{|D|}\mathrm{Gini}(D_r).$$

Since r^* minimizes the above expression, it must hold that:

$$\mathbb{E}[\mathrm{Gini}(D_l, D_r)] \leq \mathrm{Gini}(D).$$

Equality occurs precisely when no effective class separation occurs, resulting in no impurity reduction. □

Theorem 4.2 (Generalization Error Bound)**.** *Consider a decision tree T constructed using LLMT of maximum depth d from a finite rule set $\mathcal{R}$ with size K. Given n training samples, for any $\delta > 0$, with probability at least $1 - \delta$, the generalization error $R(T)$ satisfies:*

$$R(T) \;\leq\; \hat{R}(T) \;+\; \sqrt{\frac{2\Big(\log P(K, 2^d - 1) + \log\frac{2}{\delta}\Big)}{n}} \quad (4)$$

$$\leq\; \hat{R}(T) \;+\; \sqrt{\frac{2\Big((2^d - 1)\log K + \log\frac{2}{\delta}\Big)}{n}}, \quad (5)$$

where $P(K, 2^d - 1) = K!/(K - (2^d - 1))!$.

Proof. Let $N_{\mathrm{int}} = 2^d - 1$, which is the number of internal nodes of a tree with depth d. We first count the number of possible trees produced by LLMT. At each internal node, the algorithm selects a distinct rule from the global pool $\mathcal{R}$ of size K, without replacement. Thus, the number of possible ordered selections of N_{int} distinct rules is

$$P(K, N_{\mathrm{int}}) \;=\; \frac{K!}{(K - N_{\mathrm{int}})!}.$$

Each selection uniquely determines the split structure of the tree, since the structure is fixed (binary tree of maximum depth d) and rules are not reused globally.

Next, we consider leaf labels. In LLMT, leaf predictions are determined by the *leaf-refinement step*: given the path of rules leading to the leaf, the LLM outputs a confidence distribution, and the label is set using a fixed threshold τ. This procedure depends only on the rule path and τ, not on the training data. Therefore, once the split structure is fixed, the leaf labels are fixed as well, and there is no additional multiplicative factor for label assignments.

Let $\mathcal{H}$ be the set of all trees that can be produced by the algorithm. Then $|\mathcal{H}| \leq P(K, N_{\mathrm{int}})$. For each tree $T \in \mathcal{H}$, let $\hat{R}(T)$ be its empirical error on n i.i.d. samples and $R(T)$ its population error. By Hoeffding's inequality,

$$\Pr\big(|R(T) - \hat{R}(T)| \geq \varepsilon\big) \;\leq\; 2\, e^{-2n\varepsilon^2}.$$

Applying the union bound over all $T \in \mathcal{H}$ gives

$$\Pr\Big(\sup_{T \in \mathcal{H}} |R(T) - \hat{R}(T)| \geq \varepsilon\Big) \;\leq\; 2\,|\mathcal{H}|\, e^{-2n\varepsilon^2}.$$

Setting the right-hand side to δ and solving for ε yields

$$\varepsilon \;=\; \sqrt{\frac{2\big(\log|\mathcal{H}| + \log(2/\delta)\big)}{n}} \quad (6)$$

$$\leq\; \sqrt{\frac{2\big(\log P(K, N_{\mathrm{int}}) + \log(2/\delta)\big)}{n}}. \quad (7)$$

This proves the first inequality in (5). The second inequality follows from the fact that

$$\log P(K, N_{\mathrm{int}}) = \sum_{t < N_{\mathrm{int}}} \log(K - t) \leq N_{\mathrm{int}} \log K.$$

□

B Algorithm

We summarize the three-stage LLMT framework in Algorithm 1 and its optional ensemble extension, LLMT Forest, in Algorithm 2. Algorithm 1 presents rule generation (lines 1–2), tree assembly (lines 3–12), and leaf refinement (lines 13–20), while Algorithm 2 describes random feature-subspace sampling, bootstrap sampling, independent LLMT tree construction, and majority-vote aggregation.

Algorithm 1: The process of LLMT

Input: Training samples $\mathcal{D}$, depth d, LLM $f_\theta(\cdot)$

Output: Decision Tree T

Step 1 - Rule Generation:
$\mathcal{R} \leftarrow f_\theta(P_{rule})$ where $\mathcal{R} = [(r_k, s_k)]_{k=1}^{K}$
Step 2 – Tree Assembly:
for $i = 1, \ldots, d-1$ **do**
  for $j = 2^{i-1}, \ldots, 2^i - 1$ **do**
    $R_{\text{path}} \leftarrow \{\text{rules used until node } j\}$;
    $\mathcal{R}_{\text{candidate}} \leftarrow \mathcal{R} \setminus R_{\text{path}}$ (group by s_k);
    for $G \in \mathcal{R}_{\text{candidate}}$ **do**
      select $r^* \in G$ by Eq. (1) on $\mathcal{D}_j$
      if Gini-gain$(r^*, \mathcal{D}_j) > 0$ **then**
        $T[j] \leftarrow r^*$;
        break;
Step 3 – Leaf Refinement:
foreach leaf node $T[i] \in T$ **do**
  $C_p \leftarrow f_\theta(T[i]; P_{refine})$
  $y^* \leftarrow \arg\max \mathbf{C_p}$
  $c^* \leftarrow \max \mathbf{C_p}$
  if $c^* > \tau \ \wedge \ y^* \neq T[i]$ **then**
    $T[i] \leftarrow y^*$
return T;

Algorithm 2: The process of LLMT Forest

Input: Training samples $\mathcal{D}$, feature set $\mathcal{F}$, number of trees M, depth d, LLM $f_\theta(\cdot)$

Output: LLMT Forest $\mathcal{T}$

$q \leftarrow \left\lceil \sqrt{|\mathcal{F}|} \right\rceil$
for $m = 1, \ldots, M$ **do**
  $\mathcal{F}_m \leftarrow \text{RandomSubset}(\mathcal{F}, q)$
  $\mathcal{D}_m \leftarrow \text{Project}_{\mathcal{F}_m}\big(\text{Bootstrap}(\mathcal{D})\big)$
  $T_m \leftarrow \text{LLMT}(\mathcal{D}_m, d, f_\theta(\cdot))$
$\mathcal{T}(x) \leftarrow \text{MajorityVote}\big(\{T_m(x|_{\mathcal{F}_m})\}_{m=1}^{M}\big)$
return $\mathcal{T}$

C Theoretical Time Complexity Analysis

In practice, dominant runtime cost of our method comes from LLM calls; the tree-assembly steps is lightweight and takes less than 5% of the total wall-clock time across all datasets. Here we provide a theoretical analysis of the time complexity.

Let n be the number of training examples, K the number of generated rules, $d_{\max}$ the maximum tree depth, and T_{LLM} the time of a single LLM call.

Rule Generation and Leaf Refinement: For rule generation, we need to generate rules for each internal node. In the worst case, a complete binary tree of depth $d_{\max}$ has $2^{d_{\max}} - 1$ internal nodes, requiring at most $2^{d_{\max}} - 1$ LLM calls. For leaf refinement, we need to evaluate each leaf node, which requires at most $2^{d_{\max}}$ LLM calls (one per leaf). Therefore, the total cost for LLM calls is at most $(2^{d_{\max}} + 1)T_{\text{LLM}}$.

Tree Assembly: For tree assembly, at each internal node, we evaluate at most K rules on its local subset of examples. Since each example appears in at most $d_{\max}$ nodes along its path from root to leaf, the total number of rule evaluations is at most $K \cdot n \cdot d_{\max}$. Each rule evaluation involves checking the condition against an example, which takes constant time. Therefore, the total cost for tree assembly is $O(Knd_{\max})$.

Total Time Complexity: Combining both components, the total time complexity is:

$$(2^{d_{\max}} + 1)T_{\text{LLM}} + O(Knd_{\max}).$$

Since T_{LLM} is typically much larger than the time for a single rule evaluation, and K is set to $\max(10, 2^{d_{\max}} - 1)$ (as described in Section 5), the LLM call cost dominates the overall runtime in practice.

D Step-by-Step Example of LLMT

We provide a concrete example below to illustrate how LLMT works step-by-step. This example uses the Diabetes dataset with 4 training samples, maximum depth 3, and refinement threshold $\tau = 0.7$.

Training samples (toy subset):

- Sample 1: [Glucose=78, BMI=31.2, Age=42, DPF=0.382, ...] → no
- Sample 2: [Glucose=64, BMI=29.2, Age=21, DPF=0.192, ...] → no
- Sample 3: [...] → yes
- Sample 4: [...] → yes

Step 1 – Rule generation.

The LLM, given only task metadata, outputs meta-rules with confidence scores, e.g.:

- `Glucose < 140 [confidence: 10]`

- `BMI < 30.0 [confidence: 9]`
- `Age < 30 [confidence: 8]`
- `DPF < 0.5 [confidence: 8]`
- ...

Step 2 & 3 – Tree assembly + leaf refinement. At the root, we evaluate candidate rules by Gini gain and select `BMI < 30.0`. This splits the data into a pure left node (1 sample, class: no) and a mixed right node (3 samples). The left node becomes a leaf; we send its path (`BMI < 30.0`) to the LLM, which returns calibrated probabilities (e.g., no: 0.700, yes: 0.300), so the leaf remains `no` since confidence $\geq \tau$. For the right node, we again select the rule with the highest Gini gain (e.g., `DPF < 0.5`) and recurse until either purity or depth constraints are met. For each resulting leaf path (e.g., `BMI` $\geq$ `30.0 AND DPF` $\geq$ `0.5`), we perform the same refinement step via the LLM and only change the leaf label when confidence exceeds τ.

The final toy tree has the form of Figure 7:

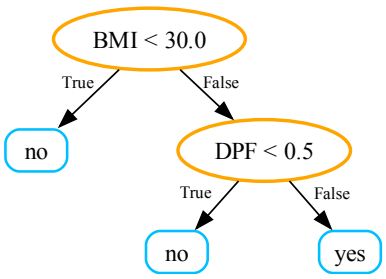


Figure 7: LLMT Example Tree (Step-by-Step)

E Information of Datasets

The details of the evaluated datasets are presented below: 1) **Nursery** (Rajkovic, 1989): A multi-class classification dataset originally used to evaluate nursery school application outcomes; 2) **Diabetes** (Smith et al., 1988): A binary classification dataset from the Pima Indians Diabetes Database to predict the onset of diabetes; 3) **Spambase** (Hopkins et al., 1999): A binary classification dataset from the UCI repository to determine whether an email is spam; 4) **Abalone** (Nash et al., 1994): A binary dataset aiming to predict the age of abalones (measured by the number of rings); 5) **Blood** (Yeh, 2008): A binary classification dataset on blood donation behavior to predict whether a donor will give blood in the future based on historical donation patterns; 6) **Iris** (Fisher, 1936): A classical multi-class dataset consisting of flower measurements for classifying iris species; 7) **Breast** (Patrício et al., 2018): A binary classification dataset using resistin, glucose, age, and BMI to predict breast cancer presence; 8) **Glioma** (Tasci et al., 2022): A binary classification dataset for glioma grading based on clinical and molecular characteristics; 9) **Communities** (Redmond, 2002): A high-dimensional classification dataset with 103 features for predicting violent-crime risk tiers; 10) **Ecom** (Rubachev et al., 2025): A 119-feature coupon-redemption dataset derived from the Acquire Valued Shoppers data using the TabReD preprocessing pipeline; 11) **Myocardial** (Golovenkin et al., 2020): A 111-feature clinical dataset for identifying chronic heart failure after myocardial infarction. Among them, we select Diabetes, Spambase, and Nursery as the major datasets for evaluation across various settings. These datasets vary widely in sample size, feature dimensionality, and number of classes, providing a comprehensive testbed.

We list all the dataset information in Table 6. All datasets are publicly available. The UCI-hosted datasets used in this paper are released under the Creative Commons Attribution 4.0 license (CC BY 4.0), which permits reuse and redistribution with proper attribution.

Dataset	#Instances	#Features	#Classes
Nursery	12,960	8	3
Diabetes	768	8	2
Spambase	4,601	18	2
Abalone	4,177	8	2
Blood	748	4	2
Iris	150	4	3
Breast	116	9	2
Glioma	839	23	2
Communities	1,994	103	3
Ecom	160,057	119	2
Myocardial	686	111	2

Table 6: Statistics of datasets used in our experiments.

F Experimental Settings

This section provides adequate experimental parameters for reproducibility. All hyperparameters, model configurations, and experimental settings are detailed below.

F.1 Experimental Details for Observation 1 E

In Observation 1, we compare LLM performance under two scenarios: *original* and *masked*. In the *original* scenario, the model has access to the full dataset description, including feature names and label names. For example, a feature appears as `Age` or `Glucose` with its semantic meaning. In the *masked* scenario, the model does not have access to feature names or label names. Instead, features are represented only as column numbers (e.g., `Feature 2` or `Column 3`), and labels are similarly anonymized (e.g., `Class 0` and `Class 1` instead of `yes` and `no`). This masking removes the semantic context that LLMs rely on for leveraging their background knowledge about the task domain.

To reduce dependence on old, widely used benchmarks, we extend the same original-versus-masked protocol to six newer datasets. Breast (2018) and Glioma (2022) already appear in our main evaluation. Paddy was released by UCI in July 2025 (Subramaniyan, 2023), and Students was released in July 2025 (Hossen and Uddin, 2025); both postdate Llama-4 Maverick, the newest model in this experiment. Loan and Personality are synthetic datasets from the 2025 Kaggle Playground Series (Kaggle, 2025b,a), so their generated records could not have appeared in model pretraining. We evaluate the same four LLMs used in Figure 1. Table 7 reports accuracy and the absolute percentage-point reduction caused by masking.

Masking lowers accuracy in 22 of 24 comparisons, leaves one unchanged, and improves one, with an average reduction of 20.14 percentage points. The pattern on post-cutoff and synthetic datasets reduces the likelihood that the effect is explained only by memorization of old benchmark records. Nevertheless, masking changes the semantic information available to the model and therefore cannot by itself prove the absence of pretraining memorization.

F.2 Experimental Details for Observation 2

We compare rule sets and path sets with the same total number of atomic feature conditions. Under a condition budget n, a rule set contains n independent single-condition rules, whereas a path set contains one or more conjunctive paths with n conditions in total. The two prompts use identical features, labels, and comparison operators. We use Qwen2.5-72B with temperature 0.8 and evaluate condition budgets $n \in \{3, 4\}$.

Let $\{(x_i, y_i)\}_{i=1}^{N}$ be the evaluation set and S a generated rule or path set. A member $s \in S$ is triggered when x_i satisfies its condition or all conditions along its path, and predicts label $\hat{y}_s$. We define

$$T_i(S) = \{s \in S : s \text{ is triggered by } x_i\},$$
$$C_i(S) = \{s \in T_i(S) : \hat{y}_s = y_i\}.$$

Let $t_i = |T_i(S)|$ and $c_i = |C_i(S)|$. Set Utility and Set Accuracy are computed as

$$\text{Utility}(S) = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[c_i > 0],$$

and

$$\text{Accuracy}(S) = \frac{1}{N} \sum_{i=1}^{N} a_i(S),$$
$$a_i(S) = \begin{cases} c_i/t_i, & t_i > 0, \\ 0, & t_i = 0. \end{cases}$$

Set Utility measures whether an instance is covered by at least one correct member, whereas Set Accuracy measures the proportion of correct predictions among triggered members and assigns zero when none is triggered. We average both metrics over 100 generations for each dataset, set type, and condition budget.

Table 8 shows that rule sets achieve higher mean Set Utility and Set Accuracy than path sets under both budgets, indicating that the advantage persists when the total number of atomic conditions is controlled.

F.3 Dataset-Specific Parameters

Table 9 lists dataset-specific parameters. Max Depth (3 or 4, depending on dataset) is shared by all methods. Test Size is 30 for Breast dataset and 100 for others. The n_estimators column applies to XGBoost, Random Forest and LLMT Forest.

F.4 LLM Configuration Settings

Table 10 details the LLM API configuration used for all LLM-based methods. The same Qwen2.5-72B backbone is used for all baselines. Request intervals and timeouts are operational parameters adjusted to dataset workload and provider status rather than fixed experimental hyperparameters.

Dataset	Model	Original	Masked	Δ	Dataset	Model	Original	Masked	Δ
Breast	DeepSeek-V3	56.0	50.0	6.0	Paddy	DeepSeek-V3	34.5	31.5	3.0
Breast	Llama-4	56.9	49.1	7.8	Paddy	Llama-4	40.0	37.5	2.5
Breast	Gemma-2	55.2	52.6	2.6	Paddy	Gemma-2	37.0	32.5	4.5
Breast	Qwen2.5	54.3	54.3	0.0	Paddy	Qwen2.5	32.5	30.5	2.0
Glioma	DeepSeek-V3	89.0	34.0	55.0	Personality	DeepSeek-V3	93.0	59.0	34.0
Glioma	Llama-4	89.0	35.0	54.0	Personality	Llama-4	93.0	64.0	29.0
Glioma	Gemma-2	53.0	57.0	−4.0	Personality	Gemma-2	93.0	61.5	31.5
Glioma	Qwen2.5	85.5	33.0	52.5	Personality	Qwen2.5	93.5	81.5	12.0
Loan	DeepSeek-V3	68.0	52.5	15.5	Students	DeepSeek-V3	77.0	56.0	21.0
Loan	Llama-4	68.0	49.5	18.5	Students	Llama-4	72.5	39.0	33.5
Loan	Gemma-2	54.5	44.0	10.5	Students	Gemma-2	66.5	43.5	23.0
Loan	Qwen2.5	70.5	34.0	36.5	Students	Qwen2.5	78.5	46.0	32.5

Table 7: Original-versus-masked zero-shot accuracy (%) on six newer datasets. Δ is Original minus Masked in percentage points. Llama-4 denotes Llama-4 Maverick, Gemma-2 denotes Gemma-2-27B-IT, and Qwen2.5 denotes Qwen2.5-72B-Instruct.

	Condition budget $n = 3$				Condition budget $n = 4$			
Dataset	Rule U.	Path U.	Rule A.	Path A.	Rule U.	Path U.	Rule A.	Path A.
Nursery	31.87	28.32	24.75	25.86	41.34	41.35	31.20	33.32
Diabetes	35.28	32.21	32.60	29.48	64.52	65.14	50.62	50.29
Spambase	54.55	41.06	45.44	36.64	60.42	53.63	52.04	47.74
Abalone	33.41	29.68	33.05	29.45	53.11	32.23	51.42	32.05
Blood	40.01	35.45	33.92	32.03	64.24	31.66	43.48	24.49
Iris	74.73	60.81	65.18	54.84	73.51	69.10	63.72	63.45
Breast	38.99	36.28	36.77	35.66	51.82	18.39	44.57	18.12
Glioma	65.94	65.88	60.76	62.94	75.96	50.17	68.29	46.43
Mean	**46.85**	41.21	**41.56**	38.36	**60.62**	45.21	**50.67**	39.49

Table 8: Rule-set and path-set results under matched condition budgets (%). U. and A. denote Set Utility and Set Accuracy. Each entry is averaged over 100 generations.

F.5 Method-Specific Settings

Table 11 provides hyperparameters for all methods. The max_depth settings match those in Table 9 for each dataset. The n_estimators values are dataset-specific and listed in Table 9. All LLM-based methods (IO-Tree, CoT-Tree, ToT-Tree, LLMT) use the LLM configuration specified in Table 10. It is worth noting for LLMT and LLMT Forest, confidence groups are formed by grouping meta-rules with the same confidence value together.

F.6 Experimental Running Parameters

Table 12 lists general experimental running parameters. Number of Trials per Setting means that for each training sample size, we randomly sample 10 different training sets and run experiments, i.e., build 10 trees.

F.7 Implementation of two Zero-shot Baselines

To further evaluate the effectiveness and efficiency of our proposed LLMT framework, we compare it with two representative zero-shot LLM-based decision tree baselines: (1) Direct-ZSDT (Knauer et al. (2025)); (2) Step-ZSDT (Carrasco et al. (2025)). For baseline Direct-ZSDT, we reuse the official code and ensure identical prompt and parameter settings to guarantee fair comparison. For baseline Step-ZSDT, whose code is not released, we follow the prompt templates and methodology described by the authors and set the probability threshold to 0.9 as specified in the original paper.

Dataset	Max Depth	n_estimators
Diabetes	3	3
Nursery	3	3
Spambase	4	6
Abalone	3	3
Blood	3	3
Iris	3	3
Breast	3	3
Glioma	4	6
Communities	4	30
Ecom	3	5
Myocardial	3	16

Table 9: Dataset-specific parameters for all experiments.

Parameter	Value
Model	Qwen2.5-72B-Instruct
API Provider	TogetherAI
Temperature	0.0
Max Tokens	2048
Parallel Batch Size	6
Presence Penalty	0.0
Frequency Penalty	0.0
Random Seed	42

Table 10: LLM API configuration settings used for all LLM-based methods.

G Additional Experimental Results

G.1 Accuracy

The mean accuracy and standard deviation across different #shots are presented in Figure 8.

G.2 Comparisons with Additional Baselines

Tables 13, 14, and 15 present expanded comparisons with tree-ensemble and pretrained black-box baselines.

G.3 Results on High-Dimensional Datasets

Non-ensemble and ensemble methods are reported in Tables 16 and 17, respectively. Step-ZSDT is omitted because its per-feature, per-node LLM queries are prohibitively expensive in high dimensions. Overall, LLMT performs best among non-ensemble methods, achieving the highest average accuracy. LLMT Forest achieves the highest average accuracy among ensemble methods.

G.4 Interpretability Analyses

G.4.1 Rule Diversity Analysis

We ensure rule diversity through careful prompt design. Our meta-rule generation prompt (see Figures 17 and 18 for prompt details) encourages diversity through the following constraints:

- **No Redundancy**
- **Maximize Purity**

To quantify feature coverage, we generate 10 meta-rule sets for each of the eight original datasets and count the distinct features used. We use the same rule-pool size K as in the main experiments (Table 11). Table 18 reports both the number and proportion of covered features.

Coverage is 100% on six datasets, 83.3% on Spambase, and 64.8% on the 23-feature Glioma dataset. Coverage is not itself an interpretability score—a smaller, focused rule set may be easier to inspect—but these values show that rule generation is not restricted to a single dominant feature.

G.4.2 Visualized Tree Examples

Figures 9–16 provide representative Diabetes trees from every compared tree generator. The LLMT example uses glucose as its principal decision feature and exposes every threshold and leaf prediction directly. These examples complement the aggregate impurity and tree-size results in Section 5 by allowing readers to inspect the learned decision logic, while we avoid treating a single medically plausible example as quantitative evidence of clinical validity.

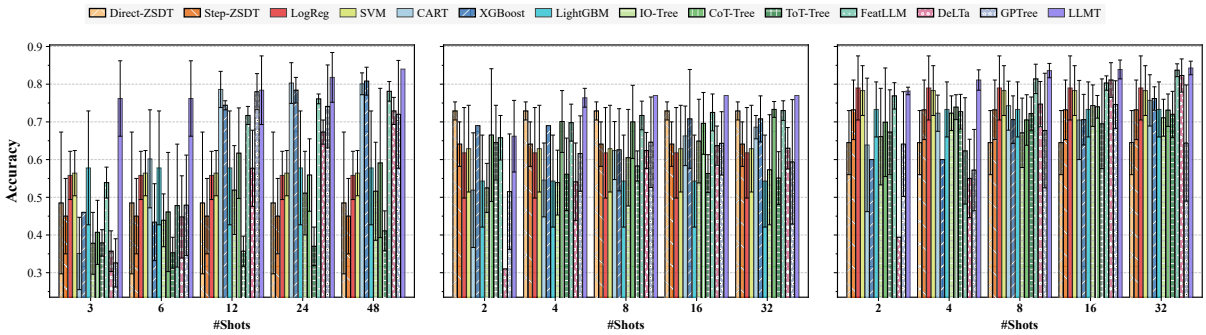
G.5 Sensitivity to Maximum Tree Depth

During parameter tuning, we examine the effect of varying maximum depth in the tree-building process; results are shown in Table 19. Performance proves sensitive to this hyperparameter. On low-dimensional datasets (e.g., Diabetes), even shallow trees (depth = 2-3) yield stable accuracy. By contrast, on higher-dimensional datasets with numerous categorical features (e.g., Nursery), deeper trees (depth = 4) consistently achieve better splits and higher accuracy. Therefore, careful tuning of maximum depth is essential - deeper trees better capture complex feature interactions when feature diversity is high.

Method	Parameter	Value
Tree methods	Maximum depth	Table 9
Tree ensembles	Ensemble size	Table 9
Step-ZSDT	Leaf-stopping threshold τ	0.9
XGBoost	`learning_rate`	0.1
LogReg	`max_iter`	1000
Linear SVM	Estimator	`LinearSVC`
	`max_iter`	10000
LightGBM	`learning_rate`	0.1
CatBoost	`depth`	Table 9
	`learning_rate`	0.1
ToT-Tree	Candidate rules per node	5
	Voting rounds	5

Method	Parameter	Value
FeatLLM	Rule-generation trials	5
DeLTa	Residual learner	Decision-tree regressor
	Fusion coefficient η	0.1
GPTree	Node type	CODE (our reproduction)
TabPFN	Pretraining-limit override	Enabled
TabLLM	Backbone	`bigscience/T0pp`
	Adaptation	IA3 via T-Few
LLMT	Leaf-refinement threshold τ	0.7
	Numeric split candidates per feature	10
	Requested meta-rules K	$\max(10, 2^{d_{\max}} - 1)$
LLMT Forest	Base estimator	LLMT (same settings)
	Feature partition	Randomized, balanced, and disjoint
	Aggregation	Majority vote

Table 11: Method-specific settings.



(a) Nursery (b) Diabetes (c) Spambase

Figure 8: The accuracy and standard deviation of different methods on three datasets.

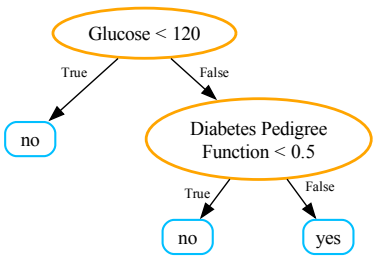


Figure 9: An Example of Direct-ZSDT

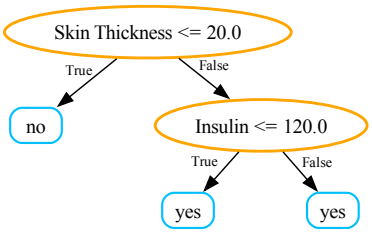


Figure 10: An Example of Step-ZSDT

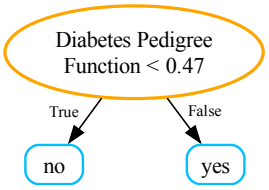


Figure 11: An Example of CART Tree

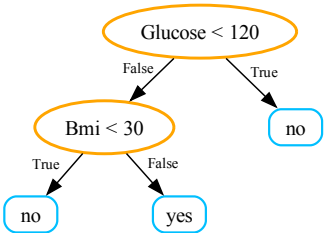


Figure 12: An Example of IO-Tree

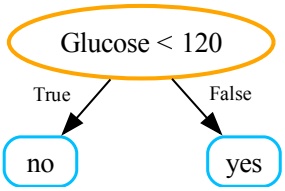


Figure 13: An Example of CoT-Tree

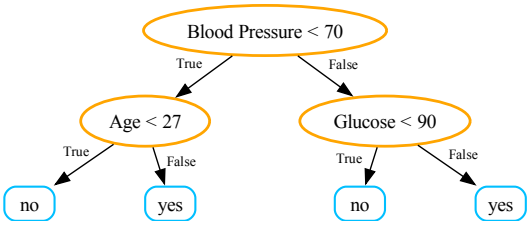


Figure 14: An Example of ToT-Tree

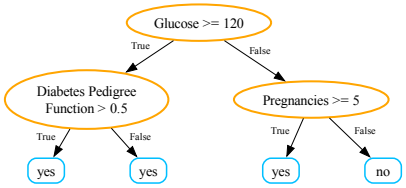


Figure 15: An Example of GPTree

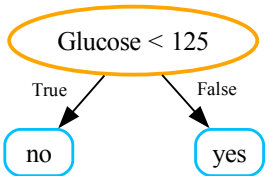


Figure 16: An Example of LLMT

Parameter	Value
Number of Trials per Setting	10
Random Seed	0
Train Batch Size	8
Test Batch Size	8

Table 12: General experimental running parameters.

H Usage of LLMs

In this work, we use ChatGPT to polish the writing of our paper.

I Prompting Templates

To facilitate application across diverse tasks, we design two prompt templates: one for generating meta-rule candidates (Figure 17 and Figure 18) and another for producing supervised probability outputs for leaf labels (Figure 19). We also include Figure 20, 21, 22, 23, 24, and 25, which respectively show the reference I/O-Tree, CoT-Tree, and ToT-Tree generation templates used as baselines in the main text. Text in blue font denotes the title of the prompt section, red font signifies variables that vary across datasets, orange font represents system commands, and text in **black** font is the fixed input text used for prompting.

Dataset	Random Forest	CatBoost	LLMT
Nursery	0.397	0.473	**0.762**
Diabetes	0.584	0.590	**0.764**
Spambase	0.725	0.743	**0.811**
Abalone	0.688	**0.700**	0.694
Blood	0.549	0.558	**0.675**
Iris	0.769	0.829	**0.940**
Breast	0.530	0.517	**0.617**
Glioma	0.733	0.759	**0.760**
Average	0.622	0.646	**0.753**

Table 13: Accuracy of Random Forest and CatBoost with two training examples per class, compared with LLMT.

Dataset	#Shots	Random Forest	CatBoost	LLMT
Nursery	3	0.349	0.464	**0.762**
	6	0.397	0.473	**0.762**
	12	0.707	0.411	**0.784**
	24	0.754	0.721	**0.818**
	48	0.798	0.740	**0.840**
Diabetes	2	0.486	0.523	**0.662**
	4	0.584	0.590	**0.764**
	8	0.659	0.650	**0.770**
	16	0.618	0.671	**0.770**
	32	0.683	0.700	**0.770**
Spambase	2	0.677	0.620	**0.782**
	4	0.725	0.743	**0.811**
	8	0.740	0.785	**0.836**
	16	0.783	0.788	**0.839**
	32	0.821	0.785	**0.843**

Table 14: Accuracy of Random Forest and CatBoost across shot counts, compared with LLMT.

Dataset	TabPFN	TabLLM	LLMT	LLMT Forest
Nursery	0.750	0.759	**0.762**	–
Diabetes	0.710	0.510	**0.764**	–
Spambase	0.802	0.673	**0.811**	–
Abalone	**0.754**	0.569	0.694	–
Blood	0.658	0.496	**0.675**	–
Iris	0.918	0.327	**0.940**	–
Breast	0.510	0.587	**0.617**	–
Glioma	**0.878**	0.521	0.760	–
Communities	0.443	N/A	0.438	**0.490**
Ecom	0.518	N/A	0.582	**0.757**
Myocardial	0.541	N/A	0.618	**0.773**

Table 15: Comparison with pretrained black-box few-shot baselines. TabLLM runs out of memory on Communities and Ecom and is prohibitively slow on Myocardial under the T0pp protocol; these cases are marked N/A.

Dataset	LogReg	SVM	CART	Direct-ZSDT	IO-Tree	CoT-Tree	ToT-Tree	FeatLLM	GPTree	LLMT (Ours)
Communities	0.424	**0.439**	0.404	0.418	0.336	0.411	0.402	0.432	0.407	0.438
Ecom	0.509	0.508	0.554	0.561	0.511	0.524	0.534	0.579	0.490	**0.582**
Myocardial	0.526	0.522	0.540	0.510	0.535	0.503	0.498	0.596	0.556	**0.618**
Average	0.486	0.490	0.499	0.496	0.461	0.479	0.478	0.536	0.484	**0.546**

Table 16: Accuracy comparison of non-ensemble methods on three high-dimensional datasets.

Dataset	XGBoost	RandomForest	LightGBM	CatBoost	DeLTa	LLMT Forest (Ours)
Communities	0.428	0.440	0.434	0.437	0.473	**0.490**
Ecom	0.695	0.504	0.412	0.503	0.619	**0.757**
Myocardial	0.702	0.545	0.581	0.535	0.565	**0.773**
Average	0.608	0.496	0.476	0.492	0.552	**0.673**

Table 17: Accuracy comparison of ensemble methods on three high-dimensional datasets.

Dataset	Diabetes	Iris	Spambase	Nursery	Abalone	Blood	Breast	Glioma
#Rules (K)	10	10	15	10	10	10	10	15
#Features	8	4	18	8	8	4	9	23
Avg. Features Used	8.0	4.0	15.0	8.0	8.0	4.0	9.0	14.9
Feature Coverage	100%	100%	83.3%	100%	100%	100%	100%	64.8%

Table 18: Rule diversity analysis: Feature coverage (Avg. Features Used / #Features) measures the proportion of features utilized in the generated rules, with higher values indicating better diversity.

Datasets	#Shots	2	3	4	5
Nursery	3	0.740±0.000	0.762±0.100	0.762±0.100	0.762±0.100
	6	0.740±0.000	0.762±0.100	0.707±0.063	0.731±0.077
	12	0.740±0.000	0.784±0.091	0.742±0.074	0.737±0.054
	24	0.740±0.000	0.818±0.066	0.713±0.084	0.735±0.045
	48	0.740±0.000	0.840±0.000	0.788±0.042	0.742±0.039
Diabetes	2	0.662±0.095	0.662±0.095	0.662±0.095	0.669±0.083
	4	0.710±0.092	0.764±0.025	0.736±0.064	0.736±0.064
	8	0.770±0.000	0.770±0.000	0.763±0.016	0.758±0.020
	16	0.770±0.000	0.770±0.000	0.763±0.016	0.756±0.024
	32	0.770±0.000	0.770±0.000	0.766±0.020	0.762±0.015
Spambase	2	0.782±0.010	0.782±0.010	0.782±0.010	0.782±0.010
	4	0.788±0.006	0.811±0.027	0.811±0.027	0.784±0.028
	8	0.788±0.006	0.843±0.028	0.836±0.019	0.810±0.023
	16	0.790±0.000	0.825±0.035	0.839±0.025	0.810±0.018
	32	0.790±0.000	0.811±0.032	0.843±0.018	0.798±0.016

Table 19: Experimental results across different maximum tree depths.

instruction Prompt

```
## Role and Task:
You are an expert data analyst specializing in meta.target or
"classification tasks". Your task is to generate high-quality meta-
rules to build a decision tree predicting the target variable
'meta.label_meaning or "output"' using the features below.
## Dataset Information:
### Features:


for feature in meta.features


    *loop.index*. feature.name: feature.desc or 'No description
available' (Type: feature.type)

if feature.is_categorical and feature.categories


    Possible values:

for cat_value, cat_desc in feature.categories.items()


    - cat_value: cat_desc

endfor



endif



endfor
### Target Variable:
meta.label_meaning or 'The output' Possible values:

for label in meta.labels

- label.nameif label.desc (label.desc)endif

endfor
## Task Requirements:
Generate exactly num_rules_required distinct and important meta-rules
for splitting the dataset into subsets with different values of the
'meta.label_meaning or "output"' label.
## Rule Format( Strict):
- For numerical features (int, float): feature name < value
- For categorical features: feature_name = category
Note: We ONLY allow '<' and '=' as numerical and categorical
operators respectively, '>' or '!=' is NOT allowed.
Important Constraints & Guidelines:
1. Confidence Score: Assign an integer from 0 (no classification
  power) to 10 (completely certain classification). Do not give 10
  confidence unless you are sure.
2. Type Matching: Use integers for int features and floating-point
  numbers for float features.
3. Decimal Precision: Use precision suited to each feature's scale —
  typically up to 3 decimals. Avoid overly precise thresholds (e.g.,
  0.165) when simpler ones (e.g., 0.22) better match value ranges.
```

Figure 17: Meta Rule Generation Prompting Template.

instruction Prompt

4. Rule Quality: Choose thresholds that create meaningful splits and align with typical value patterns. Avoid overfitting to noise; confidence < 5 only if necessary.
5. No Redundancy: Avoid trivially similar rules. Use < for numeric, = for categorical.
6. Maximize Purity: Prefer rules that create purer (more homogeneous) subgroups.
7. Score Consistency: Rules of similar quality should have similar confidence (difference ≤ 2).
8. Dominant Feature Priority: A strong feature can have MULTIPLE high-confidence rules — even HIGHER than ALL rules from weaker features.
9. First Rule Matters: Think carefully about the first rule — its feature and threshold should reflect your strongest, most confident split. It is often treated as the default decision.

Output Format (Strict):
Provide the list of rules, one per line, exactly in the specified format, sorted by confidence descending. Do NOT include any other text, explanations, or headers.

Example:
study hours per week < 3.25 [confidence: 10]
attendance rate < 74.5 [confidence: 9]
assignment completion rate < 62.75 [confidence: 8]
study hours per week < 6.0 [confidence: 7]
participation = low [confidence: 7]
attendance rate < 85.000 [confidence: 6]
previous exam grade < 59.5 [confidence: 6]

Generated Meta-Rules:

Figure 18: Meta Rule Generation Prompting Template (continued).

Leaf Refinement Prompting Template

```
## Task
Analyze whether the current prediction is reasonable based on rules,
feature descriptions, and label meanings.
Label context:

*[ label_meaning ]*
## Feature descriptions:
for desc in feature_descriptions
 *
[ desc ]
endfor
## Label meanings:
for desc in label_descriptions
 *
[ desc ]
endfor
## Instructions:
- Assume the feature values used in rules are representative of the
current sample.
- You must respect all rules and treat them as hard constraints.
- If there are multiple restrictions on the same attribute, consider
them
**together**
 (AND logic).
- For
**each possible label**
, provide a
**confidence score**
 between 0 and 1.
 0 means you're completely confident this label is incorrect for
samples matching these rules
 0.5 means the probability of this label being correct is roughly
equal to it being incorrect (maximum uncertainty)
 1 means you're completely confident this label is correct for
samples matching these rules
- The scores must
**sum to exactly 1.000**
 (representing a probability distribution).
- Use float format with
**3 decimal places**
.
## Note:
- Remember also when there is little information do not give high
probabilities (equal or higher than 0.9), unless you are very sure of
them, because you may be overestimating.
## Rules (must ALL be satisfied):
for rule in path_rules
-
*[ rule ]*
endfor
## Output format:
for label in labels
Label
*[ label.value ]*
  : <score>
endfor
Do NOT include any other text, explanations, or headers.
```

Figure 19: Leaf Refinement Prompting Template.

IO Tree Generation Prompting Template

```
### Features:

for feature in meta.features

    *loop.index*. *feature.name*: *feature.desc or 'No description
available'* (Type: feature.type)

if feature.is_categorical and feature.categories

    Possible values:

for cat_value, cat_desc in feature.categories.items()

    - cat_value: cat_desc

endfor


endif


endfor

### Target Variable:

    meta.label_meaning or 'The output'
Possible values:

for label in meta.labels

    - label.name
    if label.desc (label.desc)endif

endfor

## Label Definitions

for label in meta.labels

    - label.name: label.desc or 'No description'

endfor

## Training Samples

    Here are some examples from the training dataset:

    For each line:
     [ ', '.join(feature_names) ]  RESULT

for i, idx in enumerate(sample_indices)

    (i+1)  [ ', '.join(feature_values) ]  label_name
```

Figure 20: IO Tree Generation Prompting Template.

```
Key Requirements:

    1. Type Matching: Use integers for int features and floating-
point numbers for float features.

    2. Decimal Precision: Use precision suited to each feature's
scale — typically up to 3 decimals. Avoid overly precise thresholds
(e.g., 0.165) when simpler ones (e.g., 0.22) better match value
ranges.

    3. Feature Diversity: Use different features for each split.


Mathematical Constraints:

    - Mathematical Formalism: Let depth d ∈ [1, [ max_depth ]], for
any rule r, |r.conditions| = d-1

    - The decision tree MUST satisfy: ∀rule∈Tree, len(conditions) =
[ max_depth-1 ]
Operator Guidelines:

    1. For non-categorical features: use only '<' and '>=' conditions
(Example: persons >= 5)

    2. For categorical features: use only '<' and '=' conditions with
provided category names (Example: maintenance = high)

    3. For categorical features: Every '=' condition must have a
complementary '!=' condition for exhaustive coverage

    4. After using '=' for a categorical feature, all subsequent
rules for that feature must use '!=' conditions

    5. A categorical feature can only be assigned ONE specific '='
value in the ENTIRE tree
Tree Optimization:

    - If highly confident (>0.95) that a simple rule is sufficient,
you may generate a tree with only 1 level

    - Otherwise, use [ max_depth-1 ] levels with full splits

    - If rules already cover all labels, stop generating additional
rules

    - Avoid redundancy: e.g., (1) IF A THEN X, (2) IF NOT A THEN Y,
(3) IF B THEN X, (4) IF NOT B THEN Y - rules 3-4 are redundant
## Output Format

    BEGIN_TREE

    (1) IF condition1 AND condition2 THEN label1

    (2) IF condition1 AND condition3 THEN label2

    ...

    END_TREE

    Each rule must be a single line, start with a number in
parentheses, and follow the format: (N) IF ... THEN ...

    Do NOT use any other format. Rules must be complete and mutually
exclusive if needed.
Remember you can use [ max_depth-1 ] features at most for one rule,
and you should generate [ 2**(max_depth-1) ] rules at most.
```

Figure 21: IO Tree Generation Prompting Template (continued).

instruction Prompt

```
### Features:

% for feature in meta.features %
  *[ loop.index ]*. *[ feature.name ]*: *[ feature.desc or 'No
description available' ]* (Type: *[ feature.type ]*)

% if feature.is_categorical and feature.categories %

     Possible values:

% for cat_value, cat_desc in feature.categories.items() %
- *[ cat_value ]*: *[ cat_desc ]*

% endfor %


% endif %


% endfor %
  ### Target Variable: *[ meta.label_meaning or 'The output' ]*
  Possible values:

% for label in meta.labels %
  - *[ label.name ]*
% if label.desc %
 (*[ label.desc ]*)
% endif %


% endfor %
  ## Label Definitions

% for label in meta.labels %
- *[ label.name ]*: *[ label.desc or 'No description' ]*

% endfor %
## Training Samples
  Here are some examples from the training dataset:
  For each line:
  <*', '.join(feature_names)*> <RESULT>

% for i, idx in enumerate(sample_indices) %
(*[ i+1 ]*) *[ ', '.join(feature_values) ]* *[ label_name ]*

% endfor %
## Decision Tree RequirementsPlease generate a decision tree with a
maximum depth of *[ max_depth ]* (which means *[ max_depth-1 ]*
levels because the max_depth includes the root node).**Key
Requirements:**
  1. **Type Matching**: Use integers for int features and floating-
point numbers for float features.
  2. **Decimal Precision**: Use precision suited to each feature's
scale - typically up to 3 decimals. Avoid overly precise thresholds
(e.g., 0.165) when simpler ones (e.g., 0.22) better match value
ranges.
  3. **Feature Diversity**: Use different features for each split.
  4. **Rule Format**: Rules must follow this exact format:
     (N) IF condition [AND condition] THEN label_1**Mathematical
Constraints:**
  - Mathematical Formalism: Let depth d ∈ [1, *[ max_depth ]*], for
any rule r, |r.conditions| = d-1
  - The decision tree MUST satisfy: ∀rule∈Tree, len(conditions) = *[
max_depth-1 ]*
  - Mathematical proof required: depth=*[ max_depth ]* ⇒ each path
has
```

Figure 22: CoT Tree Generation Prompting Template.

instruction Prompt

```
exactly *[ max_depth-1 ]* splits ⇒ *[ max_depth-1 ]* conditions per
rule
  **Operator Guidelines:**
  1. For **non-categorical** features: use only '>=' and '<'
conditions (Example: persons >= 5)
  2. For **categorical** features: use only '=' and '!=' conditions
with provided category names (Example: maintenance = high)
  3. For categorical features: Every '=' condition must have a
complementary '!=' condition for exhaustive coverage
  4. After using '=' for a categorical feature, all subsequent rules
for that feature must use '!=' conditions
  5. A categorical feature can only be assigned ONE specific '='
value in the ENTIRE tree
  **Tree Optimization:**
  - If highly confident (>0.95) that a simple rule is sufficient, you
may generate a tree with only 1 level
  - Otherwise, use *[ max_depth-1 ]* levels with full splits
  - If rules already cover all labels, stop generating additional
rules
  - Avoid redundancy: e.g., (1) IF A THEN X, (2) IF NOT A THEN Y, (3)
IF B THEN X, (4) IF NOT B THEN Y - rules 3-4 are redundant
  **Operator Guidelines:**
  1. For **non-categorical** features: use only '>=' and '<'
conditions (Example: persons >= 5)
  2. For **categorical** features: use only '=' and '!=' conditions
with provided category names
  3. For categorical features: Every '=' condition must have a
complementary '!=' condition
  4. After using '=' for a categorical feature, subsequent rules for
that feature must use '!=' conditions
  5. A categorical feature can only be assigned ONE specific '='
value in the ENTIRE tree
  ## Chain-of-Thought Instructions
  You are a Decision Tree Generation Master. Your task is to analyze
the data features and generate a decision tree for classification.
  **Think Step by Step:**
  1. First, analyze the dataset characteristics and identify
potential patterns
  2. Select the best root feature and explain why it's most
informative
  3. Based on this feature, describe how to split the dataset
  4. For each subset, explain how to proceed with further splits
  5. Evaluate the effectiveness of your tree structure
  6. Ensure your final tree adheres to all mathematical constraints
  Provide your complete reasoning process in English before
presenting the final tree.
  Explain feature selection decisions, threshold choices, and
expected classification performance.
  ## Output Format
  BEGIN_TREE
  (1) IF condition1 AND condition2 THEN label1
  (2) IF condition1 AND condition3 THEN label2
  ...
  END_TREE
  Each rule must be a single line, start with a number in
parentheses, and follow the format: (N) IF ... THEN ...
  Do NOT use any other format. Rules must be complete and mutually
exclusive if needed.
  Remember you can use *[ max_depth-1 ]* features at most for one
rule, and you should generate *[ 2**(max_depth-1) ]* rules at most.
```

Figure 23: CoT Tree Generation Prompting Template (continued).

instruction Prompt (ToT)

```
## Task Description
You are a decision tree generator. Your task is to generate candidate
split rules for the current node. Please strictly follow the
requirements below. Do not provide explanations.

## Core Requirements
- Generate mutually exclusive rule pairs forming complete branches
- Directly assign final class labels to leaf nodes
- Mark intermediate nodes with [NODE]
- Always inherit parent path conditions
- When parent node depth reaches [ max_depth-1 ], child nodes must
terminate as leaves

## Structural Constraints
- Each split must form complete complementary conditions
- Continuous features use single-threshold splits, using < and >=
- Categorical features use explicit category combinations, using =
and !=
- Path conditions are automatically inherited without repetition
- Final class labels must appear in leaf node rules


% for feature in meta.features %

*[ loop.index ]*. *[ feature.name ]*: *[ feature.desc or 'No
description available' ]* (Type: *[ feature.type ]*)

% if feature.is_categorical and feature.categories %

Possible values:

% for cat_value, cat_desc in feature.categories.items() %

- *[ cat_value ]*: *[ cat_desc ]*

% endfor %


% endif %


% endfor %


## Target Variable: *[ meta.label_meaning or 'The output' ]*
Possible values:

% for label in meta.labels %

- *[ label.name ]*% if label.desc % (*[ label.desc ]*)% endif %

% endfor %


## Label Definitions

% for label in meta.labels %

- *[ label.name ]*: *[ label.desc or 'No description' ]*

% endfor %
```

Figure 24: ToT Tree Generation Prompting Template.

instruction Prompt (ToT)

```
## Training Samples
Here are some examples from the training dataset:
For each line:
<*', '.join(feature_names)*> <RESULT>


% for i, idx in enumerate(sample_indices) %

(*[ i+1 ]*) *[ ', '.join(feature_values) ]* *[ label_name ]*

% endfor %


## Examples
parent is Root->generate one (EXCEPTION: when parent is ROOT and
depth is *[ max_depth-1 ]*, generate ONLY leaves)
IF size = big THEN yes
IF size != big THEN [NODE]

parent is branch->Add AND condition
IF age >= 9 AND height < 1.3 THEN [NODE]
IF age >= 9 AND height >= 1.3 THEN no

Incorrect examples(useless split rules)
IF age >= 9 AND height < 1.3 THEN no
IF age >= 9 AND height >= 1.3 THEN no
explanation: the split rule is useless because label is always no(the
same label)

## Input Context
All possible labels:

% for label in meta.labels %

*[ label.name ]*

% endfor %


**Current Path:**
*[ display_path ]*

**Current_depth:**
*[ depth ]*

When parent node depth reaches *[ max_depth-1 ]*, child nodes MUST
terminate as leaves
```

Figure 25: ToT Tree Generation Prompting Template (continued).